PHASE 4 · ADVANCED

DAY 17

Advanced OOP

Abstract Base Classes · Metaclasses · Protocols · Dataclasses

2 hrs	4	20	Day 18
Session Length	Concept Files	Practice Problems	Next Session

Instructor Session Guide · 30-Day Python Programming Course

SESSION OVERVIEW

Goal: Move students beyond everyday OOP into the patterns used in production Python code: enforceable contracts (ABCs), classes that customize their own creation (metaclasses), structural typing (Protocol), and boilerplate-free data containers (@dataclass).

Outcome: By the end of this session, students can design an abstract base class that enforces a contract, explain what a metaclass is and write a simple one, use typing.Protocol for structural typing, and replace boilerplate classes with @dataclass — including frozen and ordered variants.

Session Timeline

Time	Segment	Focus
0:00 - 0:05	Welcome & Recap	Recap Day 16 concurrency; preview today's four topics
0:05 - 0:30	Step 1: Abstract Base Classes	abc module, @abstractmethod, enforcing contracts
0:30 - 0:55	Step 2: Metaclasses	type() as metaclass, custom metaclasses, Singleton pattern
0:55 - 1:00	Break	Short breather
1:00 - 1:25	Step 3: Protocol Classes	Structural typing, @runtime_checkable
1:25 - 1:50	Step 4: @dataclass	Auto-generated methods, frozen, order, __post_init__
1:50 - 2:00	Wrap-up & Preview	Recap, student checklist, Day 18 preview

TIP

This is a concept-dense day. Metaclasses in particular can feel abstract on first exposure — reassure students that most Python developers rarely WRITE metaclasses, but understanding them demystifies how frameworks like Django and Pydantic work under the hood.

PREREQUISITES & PREPARATION

Requirements

Requirement	Details
Python version	3.10+ (Protocol runtime_checkable behavior is stable from 3.8+, but 3.10+ recommended)
Prior sessions	Day 10-11 (OOP I & II) is essential; Day 13 (iterators) helpful context
Editor	VS Code with Pylance (will flag Protocol mismatches statically)
Files ready	day17_01 through day17_04 concept files; practice problem files

Instructor Pre-Session Checklist

■ Confirm every student can run `python --version` showing 3.10 or higher

■ Pre-load day17_01_abstract_base_classes.py in the editor for live demo

■ Have the ABC vs Protocol comparison table ready to sketch on the whiteboard

■ Test that mypy is installed if you plan to show static-checking of Protocol (optional)

STEP 1: ABSTRACT BASE CLASSES (25 min)

What to Say:

"Today we start with a question: what stops someone from creating a useless, generic PaymentGateway object that can't actually process anything? In real Python codebases, we use Abstract Base Classes to make that literally impossible — Python will refuse to let you instantiate an incomplete class."

day17_01_abstract_base_classes.py

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    def __init__(self, merchant_name):
        self.merchant_name = merchant_name

    @abstractmethod
    def process_payment(self, amount):
        pass

    @abstractmethod
    def check_status(self, transaction_id):
        pass

# Trying PaymentGateway('X') directly raises:
# TypeError: Can't instantiate abstract class PaymentGateway
#            without an implementation for abstract methods
#            'check_status', 'process_payment'
```

Live-code a PaytmGateway subclass that implements both abstract methods, then deliberately remove one method to show the TypeError again. This contrast is the core teaching moment.

Common Errors

Error	Cause	Fix
TypeError: Can't instantiate abstract class	Forgot to override an @abstractmethod	Implement every abstract method listed in the error message
Subclass instantiates fine but abstract method never called	Method was overridden but with a typo'd name	Match method names exactly, including underscores

DONE WHEN

Students can explain, in their own words, why an ABC with unimplemented abstract methods cannot be instantiated — and can fix a broken subclass by adding the missing method.

STEP 2: METACLASSES (25 min)

What to Say:

"Here's a mind-bender: in Python, classes are objects too. If classes are objects, something must have created them — and that something is called a metaclass. By default it's `type`. Today we'll write our own."

Singleton pattern via metaclass

```
class SingletonMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class AppConfig(metaclass=SingletonMeta):
    def __init__(self):
        self.settings = {'currency': 'INR', 'region': 'Mumbai'}

config1 = AppConfig()
config2 = AppConfig()
print(config1 is config2)
# True  -- both variables point to the SAME object
```

Emphasize the analogy: type() is to classes what a class is to instances. Draw the chain on the board: AppConfig instance -> AppConfig class -> SingletonMeta -> type.

Common Errors

Error	Cause	Fix
TypeError: metaclass conflict	Two base classes use incompatible metaclasses (e.g. mixing ABC and a custom metaclass incorrectly)	Make one metaclass inherit from the other, or use a single unified metaclass
Confusing __new__ vs __init__ in a metaclass	Overriding the wrong dunder for the task	Use __new__ to modify the class BEFORE creation, __call__ to intercept instantiation

DONE WHEN

Students can explain that type() is the default metaclass, and can trace through a Singleton metaclass to explain why two 'separate' instantiations return the same object.

STEP 3: PROTOCOL CLASSES (25 min)

What to Say:

"ABCs require inheritance. But what if two totally unrelated classes both happen to have a total() method? Protocol lets us describe a SHAPE instead of an ancestry — this is structural typing, or duck typing with type-checker support."

Structural typing with Protocol

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Invoiceable(Protocol):
    def total(self) -> float:
        ...

class SwiggyOrder:              # no inheritance from Invoiceable!
    def __init__(self, food_cost, delivery_fee):
        self.food_cost = food_cost
        self.delivery_fee = delivery_fee

    def total(self) -> float:
        return self.food_cost + self.delivery_fee

order = SwiggyOrder(450, 40)
print(isinstance(order, Invoiceable))
# True -- satisfied structurally, no inheritance needed
```

Sketch the ABC vs Protocol comparison table on the board (nominal vs structural typing). Ask students: 'When would you reach for Protocol over ABC?' — the answer is when classes are unrelated but share a shape.

Common Errors

Error	Cause	Fix
isinstance() raises TypeError against a Protocol	Protocol not marked @runtime_checkable	Add @runtime_checkable above the Protocol class
mypy doesn't flag a missing method	Forgot the `...` (Ellipsis) body or ran without mypy installed	Use `...` as the method body; run `mypy filename.py` to check statically

DONE WHEN

Students can define a Protocol, implement it in two unrelated classes without inheritance, and correctly predict isinstance() results with and without @runtime_checkable.

STEP 4: @dataclass DECORATOR (25 min)

What to Say:

"Last topic today, and it's a relief after metaclasses! How much boilerplate do we write just to hold data — __init__, __repr__, __eq__? @dataclass generates all of that from a few type-annotated lines."

Frozen + orderable dataclass

```
from dataclasses import dataclass, field

@dataclass(frozen=True, order=True)
class Coupon:
    discount_percent: float
    code: str = field(compare=False)
    min_order_value: float = field(default=0.0, compare=False)

    def __post_init__(self):
        if not (0 <= self.discount_percent <= 100):
            raise ValueError('discount_percent must be 0-100')

c1 = Coupon(discount_percent=10.0, code='SAVE10')
c2 = Coupon(discount_percent=50.0, code='FESTIVE50')
print(max(c1, c2))
# Coupon(discount_percent=50.0, code='FESTIVE50', min_order_value=0.0)
```

Warn students about the classic mutable-default bug: show what happens if you naively write `items: list = []` instead of `field(default_factory=list)` — every instance would SHARE the same list.

Common Errors

Error	Cause	Fix
ValueError: mutable default for field	Used a plain [] or {} as a default value directly	Use field(default_factory=list) or field(default_factory=dict)
dataclasses.FrozenInstanceError	Tried to reassign an attribute on a frozen=True dataclass	This is expected behavior; construct a new instance instead

DONE WHEN

Students can convert a regular class into a @dataclass, use default_factory correctly for mutable fields, and explain what frozen=True and order=True each do.

SESSION WRAP-UP

Student Checklist

- ■ I can explain why you can't instantiate an ABC with unimplemented abstract methods
- ■ I can describe what a metaclass is, using the 'type creates classes' analogy
- ■ I can write a Protocol and understand structural vs nominal typing
- ■ I can convert a boilerplate class into a clean @dataclass
- ■ I understand why mutable defaults need field(default_factory=...)

Key Concepts Recap

ABC: nominal typing, contract enforced at instantiation time. **Metaclass:** a class that creates classes; type is the default. **Protocol:** structural typing, no inheritance required. **@dataclass:** auto-generates __init__/__repr__/__eq__; supports frozen, order, and default_factory.

Next Day Preview

Day 18 moves into Type Hints & Static Analysis — variable/function annotations, mypy, Optional/Union, and generics with TypeVar. Today's Protocol work is the perfect on-ramp.

WATCH OUT

If students are shaky on metaclasses, that's completely normal — even experienced developers rarely write custom metaclasses day-to-day. Focus reinforcement time on ABCs and dataclasses, which they'll use constantly starting with the capstone project.

QUICK REFERENCE CARD

Day 17 Cheat Sheet

```
# Abstract Base Classes
from abc import ABC, abstractmethod
class Foo(ABC):
    @abstractmethod
    def bar(self): ...

# Metaclass basics
class MyMeta(type):
    def __new__(mcs, name, bases, ns): ...
class Foo(metaclass=MyMeta): ...

# Protocol (structural typing)
from typing import Protocol, runtime_checkable
@runtime_checkable
class HasTotal(Protocol):
    def total(self) -> float: ...

# dataclass essentials
from dataclasses import dataclass, field
@dataclass(frozen=True, order=True)
class Point:
    x: int
    y: int
    tags: list = field(default_factory=list)
```

Daily Workflow Commands

Command	Purpose
python day17_01_abstract_base_classes.py	Run the ABC concept file
python day17_practise_problems_01_abstract_base_classes.py	Run practice problems (Section A+B)
mypy day17_03_protocol_classes.py	Static-check Protocol usage (optional)

PHASE 4 · ADVANCED

DAY 17

Advanced OOP

Abstract Base Classes · Metaclasses · Protocols · Dataclasses

2 hrs	4	20	Day 18
Session Length	Concept Files	Practice Problems	Next Session

Instructor Session Guide · 30-Day Python Programming Course

SESSION OVERVIEW

Goal: Move students beyond everyday OOP into the patterns used in production Python code: enforceable contracts (ABCs), classes that customize their own creation (metaclasses), structural typing (Protocol), and boilerplate-free data containers (@dataclass).

Outcome: By the end of this session, students can design an abstract base class that enforces a contract, explain what a metaclass is and write a simple one, use typing.Protocol for structural typing, and replace boilerplate classes with @dataclass — including frozen and ordered variants.

Session Timeline

Time	Segment	Focus
0:00 - 0:05	Welcome & Recap	Recap Day 16 concurrency; preview today's four topics
0:05 - 0:30	Step 1: Abstract Base Classes	abc module, @abstractmethod, enforcing contracts
0:30 - 0:55	Step 2: Metaclasses	type() as metaclass, custom metaclasses, Singleton pattern
0:55 - 1:00	Break	Short breather
1:00 - 1:25	Step 3: Protocol Classes	Structural typing, @runtime_checkable
1:25 - 1:50	Step 4: @dataclass	Auto-generated methods, frozen, order, __post_init__
1:50 - 2:00	Wrap-up & Preview	Recap, student checklist, Day 18 preview

TIP

This is a concept-dense day. Metaclasses in particular can feel abstract on first exposure — reassure students that most Python developers rarely WRITE metaclasses, but understanding them demystifies how frameworks like Django and Pydantic work under the hood.

PREREQUISITES & PREPARATION

Requirements

Requirement	Details
Python version	3.10+ (Protocol runtime_checkable behavior is stable from 3.8+, but 3.10+ recommended)
Prior sessions	Day 10-11 (OOP I & II) is essential; Day 13 (iterators) helpful context
Editor	VS Code with Pylance (will flag Protocol mismatches statically)
Files ready	day17_01 through day17_04 concept files; practice problem files

Instructor Pre-Session Checklist

- ■ Confirm every student can run `python --version` showing 3.10 or higher
- ■ Pre-load day17_01_abstract_base_classes.py in the editor for live demo
- ■ Have the ABC vs Protocol comparison table ready to sketch on the whiteboard
- ■ Test that mypy is installed if you plan to show static-checking of Protocol (optional)

STEP 1: ABSTRACT BASE CLASSES (25 min)

What to Say:

"Today we start with a question: what stops someone from creating a useless, generic PaymentGateway object that can't actually process anything? In real Python codebases, we use Abstract Base Classes to make that literally impossible — Python will refuse to let you instantiate an incomplete class."

day17_01_abstract_base_classes.py

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    def __init__(self, merchant_name):
        self.merchant_name = merchant_name

    @abstractmethod
    def process_payment(self, amount):
        pass

    @abstractmethod
    def check_status(self, transaction_id):
        pass

# Trying PaymentGateway('X') directly raises:
# TypeError: Can't instantiate abstract class PaymentGateway
#            without an implementation for abstract methods
#            'check_status', 'process_payment'
```

Live-code a PaytmGateway subclass that implements both abstract methods, then deliberately remove one method to show the TypeError again. This contrast is the core teaching moment.

Common Errors

Error	Cause	Fix
TypeError: Can't instantiate abstract class	Forgot to override an @abstractmethod	Implement every abstract method listed in the error message
Subclass instantiates fine but abstract method never called	Method was overridden but with a typo'd name	Match method names exactly, including underscores

DONE WHEN

Students can explain, in their own words, why an ABC with unimplemented abstract methods cannot be instantiated — and can fix a broken subclass by adding the missing method.

STEP 2: METACLASSES (25 min)

What to Say:

"Here's a mind-bender: in Python, classes are objects too. If classes are objects, something must have created them — and that something is called a metaclass. By default it's `type`. Today we'll write our own."

Singleton pattern via metaclass

```
class SingletonMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class AppConfig(metaclass=SingletonMeta):
    def __init__(self):
        self.settings = {'currency': 'INR', 'region': 'Mumbai'}

config1 = AppConfig()
config2 = AppConfig()
print(config1 is config2)
# True  -- both variables point to the SAME object
```

Emphasize the analogy: type() is to classes what a class is to instances. Draw the chain on the board: AppConfig instance -> AppConfig class -> SingletonMeta -> type.

Common Errors

Error	Cause	Fix
TypeError: metaclass conflict	Two base classes use incompatible metaclasses (e.g. mixing ABC and a custom metaclass incorrectly)	Make one metaclass inherit from the other, or use a single unified metaclass
Confusing __new__ vs __init__ in a metaclass	Overriding the wrong dunder for the task	Use __new__ to modify the class BEFORE creation, __call__ to intercept instantiation

DONE WHEN

Students can explain that type() is the default metaclass, and can trace through a Singleton metaclass to explain why two 'separate' instantiations return the same object.

STEP 3: PROTOCOL CLASSES (25 min)

What to Say:

"ABCs require inheritance. But what if two totally unrelated classes both happen to have a total() method? Protocol lets us describe a SHAPE instead of an ancestry — this is structural typing, or duck typing with type-checker support."

Structural typing with Protocol

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Invoiceable(Protocol):
    def total(self) -> float:
        ...

class SwiggyOrder:              # no inheritance from Invoiceable!
    def __init__(self, food_cost, delivery_fee):
        self.food_cost = food_cost
        self.delivery_fee = delivery_fee

    def total(self) -> float:
        return self.food_cost + self.delivery_fee

order = SwiggyOrder(450, 40)
print(isinstance(order, Invoiceable))
# True -- satisfied structurally, no inheritance needed
```

Sketch the ABC vs Protocol comparison table on the board (nominal vs structural typing). Ask students: 'When would you reach for Protocol over ABC?' — the answer is when classes are unrelated but share a shape.

Common Errors

Error	Cause	Fix
isinstance() raises TypeError against a Protocol	Protocol not marked @runtime_checkable	Add @runtime_checkable above the Protocol class
mypy doesn't flag a missing method	Forgot the `...` (Ellipsis) body or ran without mypy installed	Use `...` as the method body; run `mypy filename.py` to check statically

DONE WHEN

Students can define a Protocol, implement it in two unrelated classes without inheritance, and correctly predict isinstance() results with and without @runtime_checkable.

STEP 4: @dataclass DECORATOR (25 min)

What to Say:

"Last topic today, and it's a relief after metaclasses! How much boilerplate do we write just to hold data — __init__, __repr__, __eq__? @dataclass generates all of that from a few type-annotated lines."

Frozen + orderable dataclass

```
from dataclasses import dataclass, field

@dataclass(frozen=True, order=True)
class Coupon:
    discount_percent: float
    code: str = field(compare=False)
    min_order_value: float = field(default=0.0, compare=False)

    def __post_init__(self):
        if not (0 <= self.discount_percent <= 100):
            raise ValueError('discount_percent must be 0-100')

c1 = Coupon(discount_percent=10.0, code='SAVE10')
c2 = Coupon(discount_percent=50.0, code='FESTIVE50')
print(max(c1, c2))
# Coupon(discount_percent=50.0, code='FESTIVE50', min_order_value=0.0)
```

Warn students about the classic mutable-default bug: show what happens if you naively write `items: list = []` instead of `field(default_factory=list)` — every instance would SHARE the same list.

Common Errors

Error	Cause	Fix
ValueError: mutable default for field	Used a plain [] or {} as a default value directly	Use field(default_factory=list) or field(default_factory=dict)
dataclasses.FrozenInstanceError	Tried to reassign an attribute on a frozen=True dataclass	This is expected behavior; construct a new instance instead

DONE WHEN

Students can convert a regular class into a @dataclass, use default_factory correctly for mutable fields, and explain what frozen=True and order=True each do.

SESSION WRAP-UP

Student Checklist

- ■ I can explain why you can't instantiate an ABC with unimplemented abstract methods
- ■ I can describe what a metaclass is, using the 'type creates classes' analogy
- ■ I can write a Protocol and understand structural vs nominal typing
- ■ I can convert a boilerplate class into a clean @dataclass
- ■ I understand why mutable defaults need field(default_factory=...)

Key Concepts Recap

ABC: nominal typing, contract enforced at instantiation time. **Metaclass:** a class that creates classes; type is the default. **Protocol:** structural typing, no inheritance required. **@dataclass:** auto-generates __init__/__repr__/__eq__; supports frozen, order, and default_factory.

Next Day Preview

Day 18 moves into Type Hints & Static Analysis — variable/function annotations, mypy, Optional/Union, and generics with TypeVar. Today's Protocol work is the perfect on-ramp.

WATCH OUT

If students are shaky on metaclasses, that's completely normal — even experienced developers rarely write custom metaclasses day-to-day. Focus reinforcement time on ABCs and dataclasses, which they'll use constantly starting with the capstone project.

QUICK REFERENCE CARD

Day 17 Cheat Sheet

```
# Abstract Base Classes
from abc import ABC, abstractmethod
class Foo(ABC):
    @abstractmethod
    def bar(self): ...

# Metaclass basics
class MyMeta(type):
    def __new__(mcs, name, bases, ns): ...
class Foo(metaclass=MyMeta): ...

# Protocol (structural typing)
from typing import Protocol, runtime_checkable
@runtime_checkable
class HasTotal(Protocol):
    def total(self) -> float: ...

# dataclass essentials
from dataclasses import dataclass, field
@dataclass(frozen=True, order=True)
class Point:
    x: int
    y: int
    tags: list = field(default_factory=list)
```

Daily Workflow Commands

Command	Purpose
python day17_01_abstract_base_classes.py	Run the ABC concept file
python day17_practise_problems_01_abstract_base_classes.py	Run practice problems (Section A+B)
mypy day17_03_protocol_classes.py	Static-check Protocol usage (optional)